Contents

Chapter 1: Introduction

1.1 What this course will teach

This workshop will introduce the student to Python coding, electronics, and project design. We will be building several projects ranging from simple (less components and simpler code) to more complex (more components or more complicated code). These projects are based on the ESP32-C3 microcontroller (specifically the development board produced by Seeed Studio) which is running MicroPython and they depend on some other electronics components such as LEDs, buttons, and more.

Students are encouraged to make use of the written instructions and diagrams and to explore and play with the components for themselves to see how they work and what they do. There are many online resources as well for learning.

1.2 Project Kit Contents

[photo placeholder]

Figure 1.1: This image shows all of the contents of the project kit and labels what each one is

1.3 Working with the microcontroller

1.3.1 Installing the driver

Connecting the microcontroller to your computer may require the installation of a USB driver. If so, download the driver for your OS from this page and install as instructed: `https://ftdichip.com/drivers/vcp-drivers/`. Once installed successfully, unplug and replug the USB cable into the microcontroller and then press the small reset button on the microcontroller board labeled "R" (for reset).

1.3.2 Pinout Reference

On any integrated circuit, that is a chip that contains electronic logic, a pinout is a diagram and description of what each of the pins of that circuit are used for. This is a very useful document for any electronics engineer to have handy and to reference. The pinout diagram for the microcontroller that we are using as part of this workshop is replicated below:

[photo placeholder]

Figure 1.2: A simplified pinout of the Seeed Studio ESP32-C3 development board

In this diagram, you'll notice several things:

- An image of the microcontroller board in the center
- A label for each pin indicating its function

In the code throughout this guide, you will see references to pin numbers like **Pin(5)**. Wherever you see this, it means that the code is expecting a connection to, in this example, the 4th pin down on the left. If you find that your code is not working as expected, double check the pin numbers in the code vs. the placement of your wires.

1.3.3 Project Guide Updates

In case you are reading an out of date version of this guide, you can find the latest at `https://netapp-ptc.github.io/YWIT-2026/project_guide.pdf`.

You can also find the source code for all of the workshops and other useful information in the GitHub repository at `https://github.com/NetApp-PTC/YWIT-2026`.

Chapter 2: Electronics Essentials

Electronics is a fascinating world. This beginner's guide will introduce you to the essential components and tools you'll need to start building your own simple circuits. We'll cover the basics of breadboards, LEDs, buttons, resistors, and jumper cables. You should feel free to explore on your own as well using the components in your kit. While this guide will be useful, it is not even close to everything you'll need to know. There is always more to learn and if electronics interests you, there are lots of online tutorials as well as formal education to guide you further.

2.1 The Breadboard: Your Circuit's Foundation

2.1.1 What is a breadboard?

A breadboard is a reusable platform for prototyping electronic circuits without the need for soldering. It's a plastic board with numerous tiny holes arranged in rows and columns. These holes are connected internally, making it easy to connect components together.

2.1.2 Understanding the layout

Power rails

The long horizontal rows along the top and bottom edges are typically used as power rails to distribute power (positive and negative) throughout the circuit.

Internal connections

The remaining rows are divided into vertical columns, with each column having five holes connected internally. These connections allow you to easily connect multiple components within the same column.

[photo placeholder]

2.2 LEDs: Lighting Up Your Circuits

2.2.1 What is an LED?

An LED (Light-Emitting Diode) is a semiconductor device that emits light when an electric current passes through it. They are available in various colors and are commonly used as indicators or displays in electronic circuits.

2.2.2 Polarity

LEDs have two leads:

- **Anode (longer lead):** Connected to the positive (+) side of the power supply.
- **Cathode (shorter lead):** Connected to the negative (-) side of the power supply.

[photo placeholder]

2.2.3 Current limiting resistor

It's crucial to connect a resistor in series with an LED to limit the current flowing through it. Otherwise, the LED may get damaged or burn out. The value of the resistor depends on the LED's specifications and the power supply voltage.

2.3 Buttons: Interacting with Your Circuits

2.3.1 What is a button?

A button, also known as a pushbutton switch, is a simple mechanical device that completes or breaks an electrical circuit when pressed. They are commonly used to trigger actions or provide input to electronic circuits.

2.3.2 Types of buttons

- **Normally open (NO):** The circuit is open (no connection) when the button is not pressed and closes (connection made) when pressed.
- **Normally closed (NC):** The circuit is closed (connection made) when the button is not pressed and opens (no connection) when pressed.

2.3.3 Connected Pins

Many buttons will have more than two pins (often 4). If they do, then often you can see on the bottom that some of those pins will already be connected together. Pay attention to this so that you wire it up the right way.

[photo placeholder]

2.4 Jumper Cables: Making Connections

2.4.1 What are jumper cables?

Jumper cables are short wires with connectors at both ends used to make temporary connections between components on a breadboard or between a breadboard and other devices.

2.4.2 Using jumper cables

Insert the connectors into the appropriate holes on the breadboard or device to establish a connection.

[photo placeholder]

2.5 Resistors: Controlling Current Flow

2.5.1 What is a resistor?

A resistor is a passive component that limits the flow of electric current in a circuit. They are essential for protecting components (like LEDs) from excessive current and for controlling voltage levels in circuits.

2.5.2 Resistance value

The resistance value is measured in ohms (Ω) and is indicated by colored bands on the resistor body. You can use a resistor color code chart to decode the value.

[photo placeholder]

2.6 Safety Tips:

- Always disconnect the power supply before making any changes to the circuit.
- Be careful not to short-circuit the power supply by connecting the positive and negative rails directly.
- If you smell burning or see smoke, immediately disconnect the power supply.

Chapter 3: MicroPython IDE

3.1 Connecting Your Microcontroller

Throughout this guide, we will be making use of a free online IDE for connecting our microcontroller to our computer and loading, editing, and running code on it. You can access the IDE from your browser by going to `https://viper-ide.blackhart.dev/`. Click on the USB icon in the top right and choose your microcontroller from the list:

[photo placeholder]

Figure 3.1: Click the button highlighted in red.

If you see multiple items in the dialog that pops up, choose the one that starts with "USB JTAG". See below for an example:

[photo placeholder]

Figure 3.2: Click the button highlighted in red.

Once you have connected, you will see a green dialog labeled "Device connected" and the file manager on the list will populate with the list of files installed on the device:

[photo placeholder]

Figure 3.3: Make sure to open the right file for this project

After your device is connected, refer back to the chapter you are working on for the next steps.

Chapter 4: Project 1: Blink

4.1 Overview

This project is a placeholder for Project 1.

4.2 Directions

4.2.1 Creating the circuit

4.2.2 Programming the microcontroller

Click on the file named "project_1_blink.py" and run it using the IDE. Refer back to Chapter 3 for IDE instructions.

4.3 Review

4.4 Possible Extensions

Chapter 5: Project 2: Pocket DJ

5.1 Overview

This project is a placeholder for Project 2 (Pocket DJ). Students will program pitch frequencies, rhythm loops, and an LED pulse animation synchronized to the sound using push buttons and an LED strip.

5.2 Directions

5.2.1 Creating the circuit

5.2.2 Programming the microcontroller

Click on the file named "project_2_pocket_dj.py" and run it using the IDE. Refer back to Chapter 3 for IDE instructions.

5.3 Review

5.4 Possible Extensions

Chapter 6: Project 3: Red Light, Green Light

6.1 Overview

In this project, you will build a reflex game with a button and a 3-pixel WS2812B LED strip. Each reaction window cycles through red, yellow, and green phases:

- During **RED**, do not press. Pressing during red ends the round immediately.
- During **YELLOW**, get ready for green and do not press yet.
- During **GREEN**, press the button as quickly as possible to score a point.

Each successful green press prints your reaction time in milliseconds to the serial console. Each round has 5 total reaction windows. A hit during green scores a point, and a missed green window counts as a miss.

Learning goals:

- Read a pushbutton using an internal pull-up resistor
- Control WS2812B addressable LEDs with MicroPython
- Build a simple game state machine
- Measure reaction time with millisecond timers

6.2 Components

- Seeed XIAO ESP32-C3 microcontroller
- 3x WS2812B addressable RGB LEDs wired in series
- 1x pushbutton
- Breadboard
- Jumper wires

6.3 Directions

Remove previous components

Before starting Project 3, remove components and wires from the previous project.

6.3.1 Creating the circuit

WS2812B strip wiring

Wire your 3-pixel WS2812B chain as follows:

WS2812B Pin	XIAO GPIO	XIAO Label
DIN (first LED)	GPIO 20	D7
VCC	3V3	3V3
GND	GND	GND

Button wiring

Wire a pushbutton between GPIO21 and GND:

Button Pin	XIAO GPIO	XIAO Label
One side	GPIO 21	D6
Other side	GND	GND

The code enables the internal pull-up resistor, so the pin reads HIGH when idle and LOW when pressed.

6.3.2 Programming the microcontroller

Open the file `project_3_red_light_green_light.py` and run it in the IDE (Chapter 3).
At startup, the serial console will print instructions and wait for a button press to begin:

```
Project 3: Red Light, Green Light
Press button to start.
```

During gameplay:

- The 3 LEDs act like a stoplight: red LED, yellow LED, and green LED are used individually.
- RED phase lights only the red LED and prints "RED! Do not press."
- YELLOW phase lights only the yellow LED and prints "YELLOW! Get ready..."
- GREEN phase lights only the green LED and prints "GREEN! Press now."
- Idle/round-over uses the yellow LED as a waiting indicator.
- A valid green press prints reaction time and increases score.
- A missed green window is counted as one of the 5 windows.
- A red/yellow press ends the round and prints final score + best reaction.

6.3.3 Hardware test checklist

Use this checklist to confirm your circuit and code are working:

1. Confirm LED order on your strip: first LED (DIN side) is red, middle is yellow, last is green.
2. Press button from idle state: yellow wait indicator transitions into red/green gameplay cycling.
3. Wait for GREEN (green LED only) and press quickly: score increases and a reaction time is printed.
4. Do not press during RED or YELLOW: pressing before green ends the round immediately.
5. Skip pressing during one green window: verify it is counted as a miss and the next window starts.
6. Repeat at least 5 rounds to confirm stable behavior and reliable button detection.

6.4 Review

In this project, you combined timed logic, button input, and LED output to build a full game loop. You practiced:

- State-based programming (idle, green, red, round_over)
- Reusing a shared debounced button library for cleaner input handling
- Using `time.ticks_ms()` and `time.ticks_diff()` for timing
- Providing immediate visual and text feedback to players

6.5 Possible Extensions

Try these enhancements:

- Add difficulty levels by shrinking GREEN timing windows
- Display average reaction time after each round
- Add a buzzer sound effect for valid and invalid presses
- Use different LED patterns (chase, blink, gradient) per game state
- Add a countdown before each round starts

Chapter 7: Project 4: Safe Cracker

7.1 Overview

In this project, you will build a combination-lock game using a rotary encoder and a small speaker. Spin the encoder dial to select digits 0–9, press the built-in encoder button to lock in each digit, and listen for audio hints that get warmer as you approach the correct value. Match the full secret sequence to "open the safe!"

Over the course of this project, you will:

- Wire a KY-040 rotary encoder module to your microcontroller
- Wire a small speaker using PWM (Pulse Width Modulation) to generate tones
- Use library code to reliably read encoder rotation and button presses
- Program a game loop with randomized secret codes and audio proximity feedback
- Practice reading serial output to debug your program

At the end of this project, your microcontroller will run a fully playable combination-lock game entirely driven by sound and a single knob—no screen required!

How the game works: A random 3-digit secret code (each digit 0–9) is generated when the program starts and printed to the serial console. Spin the encoder knob to change the current dial value. As you get closer to the target digit, the tone played on each tick gets higher and shorter. When you are exactly on the target, you hear a sharp high click. Press the encoder button to lock in your digit. A correct digit advances you to the next position; a wrong digit plays a low buzz and you try again. Crack all three digits to trigger the victory fanfare and start a new game!

7.2 Components

- Seeed XIAO ESP32-C3 microcontroller
- KY-040 rotary encoder module (5-pin, includes onboard pull-up resistors)
- Treedix 1W 8Ω mini speaker (JST 2-pin connector)
- Breadboard
- Jumper wires

7.3 Directions

Remove previous components

Before beginning, remove any components from prior chapters including LEDs, buttons, and wires. You may leave the microcontroller attached to the breadboard.

7.3.1 Understanding the KY-040 Rotary Encoder

A rotary encoder is a type of position sensor that converts the angular position (rotation) of a knob into electrical signals. Unlike a potentiometer, an encoder has no physical stops—you can spin it continuously in either direction.

The KY-040 module has five pins:

- **CLK** — Clock signal; pulses as the knob turns
- **DT** — Data signal; used together with CLK to determine direction
- **SW** — Switch; goes LOW when the built-in button is pressed
- **VCC** — Power supply (connect to 3.3V)
- **GND** — Ground

> **Important:** Power the KY-040 from **3.3V**, not 5V. The ESP32-C3 GPIO pins are not 5V-tolerant, so using 5V could damage your microcontroller.

7.3.2 Creating the circuit

Using jumper cables, assemble the circuit according to the following wiring table. The encoder and speaker share the breadboard with the microcontroller.

KY-040 Encoder Wiring

KY-040 Pin	XIAO GPIO	XIAO Label
CLK	GPIO 10	D10
DT	GPIO 9	D9
SW	GPIO 8	D8
VCC	3V3	3V3
GND	GND	GND

Speaker Wiring

Connect the Treedix speaker's JST connector leads to the breadboard:

Speaker Lead	XIAO GPIO	XIAO Label
Signal (+)	GPIO 20	D7
Ground (−)	GND	GND

Wiring photo placeholder

Figure 7.1: Encoder and speaker wired to the XIAO ESP32-C3

7.3.3 Understanding the Code

Open the file named `project_4_safe_cracker.py` in the IDE. Let's walk through the key sections.

Configuration Constants

At the top of the file, a configuration block defines all pin assignments and game parameters in one place. This makes it easy to adjust the game without hunting through the code:

Listing 7.1: Configuration constants

```
CODE_LENGTH = 3
DIAL_MIN = 0
DIAL_MAX = 9

PIN_CLK = 10
PIN_DT = 9
PIN_SW = 8
PIN_SPEAKER = 20

TONE_EXACT  = (880, 80)    # high click when exactly on target
TONE_WARM   = (550, 50)    # warm mid tone when 1-2 away
TONE_COLD   = (220, 30)    # low cold tone when 3+ away
TONE_BUZZ   = (150, 200)   # wrong-digit buzz
VICTORY_NOTES = [(262, 150), (330, 150), (392, 150), (523, 400)]
```

Playing Tones with PWM

The `play_tone` function uses PWM (Pulse Width Modulation) to drive the speaker. PWM rapidly switches the GPIO pin on and off at a given frequency, which the speaker cone turns into sound:

Listing 7.2: PWM tone helper

```
def play_tone(freq, duration_ms):
    pwm = PWM(Pin(PIN_SPEAKER), freq=freq, duty=512)
    time.sleep_ms(duration_ms)
    pwm.deinit()
```

A `duty` of 512 out of 1023 means the pin is HIGH roughly half the time—this is called a 50% duty cycle and produces the loudest sound for a square wave.

Proximity Hints

The game measures how close the current dial value is to the target digit using a *circular distance*. Because the dial wraps around (0 and 9 are neighbors), we take the smaller of the straight-line distance and the wrap-around distance:

Listing 7.3: Circular distance on a 0-9 dial

```
def dial_distance(current, target):
    diff = abs(current - target)
    return min(diff, 10 - diff)
```

This distance is then mapped to a tone:

Distance to target	Sound
0 (exact match)	High click (880 Hz, 80 ms)
1 or 2 away	Warm mid tone (550 Hz, 50 ms)
3 or more away	Low cold tone (220 Hz, 30 ms)

Reading the Encoder and Button

The code uses two library modules from the `lib/` folder:

- **RotaryIRQ** — reads the encoder using hardware interrupts; configured for bounded range 0–9 with automatic wrap-prevention.
- **DebouncedButton** — reads the encoder's push-button and fires a callback only once per press, filtering out electrical noise (bouncing).

Listing 7.4: Setting up the encoder and button

```
knob = RotaryIRQ(
    pin_num_clk=PIN_CLK,
    pin_num_dt=PIN_DT,
    min_val=DIAL_MIN,
    max_val=DIAL_MAX,
    reverse=True,
    pull_up=True,
    range_mode=RotaryIRQ.RANGE_BOUNDED,
)
knob.add_listener(on_dial_change)
DebouncedButton(PIN_SW, on_enter)
```

`reverse=True` makes clockwise rotation increase the displayed number (adjust to `False` if your knob spins the wrong way). `pull_up=True` enables the internal pull-up resistors so the CLK and DT pins read HIGH when idle.

Game Loop

Each time the knob moves, `on_dial_change` is called automatically by the library. It reads the current encoder value, computes the distance to the target digit, and plays the corresponding hint tone. When the encoder button is pressed, `on_enter` checks whether the current value matches the target:

- **Correct:** A confirming click plays, the dial index advances. If all digits are correct, the victory fanfare plays and a new game begins.
- **Wrong:** A low buzz plays and the player tries again on the same digit.

7.3.4 Programming the microcontroller

Connect the USB cable to the microcontroller and open the IDE. Refer back to Chapter 3 for IDE instructions if needed.

Click on the file named `project_4_safe_cracker.py`. Read through the comments and code to understand how it works, then click the blue play button to start the game.

When the program starts, watch the serial console—it will print the secret code so you (or your instructor) can verify the game is working:

```
New safe code: [3, 7, 1]
Dial 1/3: showing 5 (target 3, distance 2)
```

> **Tip:** If the tones are silent, check that the speaker's signal wire is connected to GPIO 20 and that the ground wire goes to GND. If the dial counts in the wrong direction, change `reverse=True` to `reverse=False` in the code.

7.4 Review

In this project, we combined hardware input (rotary encoder) and output (PWM speaker tones) to build a complete interactive game. We learned:

- How a rotary encoder converts physical rotation into digital signals
- How PWM can drive a speaker to play different pitches
- How to use interrupt-driven library code to read hardware without blocking the main loop
- How circular distance can create smooth "hot and cold" proximity feedback
- How to structure a game with state variables, callbacks, and a main loop

7.5 Possible Extensions

If you want to experiment further, try these ideas:

- Increase `CODE_LENGTH` to 4 or 5 digits for a harder challenge
- Add a countdown timer—if you don't crack the safe in time, the game resets
- Change the victory fanfare to play a song you recognize (look up the note frequencies!)
- Add a "lives" system: after 3 wrong guesses on the same digit, the game resets
- Print the elapsed time to the serial console when the safe is cracked

Chapter 8: Project 5: Wi-Fi Mood Lamp

8.1 Overview

This project is a placeholder for Project 5 (Wi-Fi Mood Lamp). Students will write a local web server that displays a dashboard allowing control of LED lights.

8.2 Directions

8.2.1 Creating the circuit

8.2.2 Programming the microcontroller

Click on the file named "project_5_wifi_mood_lamp.py" and run it using the IDE. Refer back to Chapter 3 for IDE instructions.

8.3 Review

8.4 Possible Extensions

Chapter 9: Project 6: ESP-NOW Hot Potato

9.1 Overview

This project is a placeholder for Project 6 (ESP-NOW Hot Potato). A multiplayer wireless game where a countdown timer is passed between boards using the ESP-NOW protocol. The student holding the "potato" when the timer hits zero loses!

9.2 Directions

9.2.1 Creating the circuit

9.2.2 Programming the microcontroller

Click on the file named "project_6_hot_potato.py" and run it using the IDE. Refer back to Chapter 3 for IDE instructions.

9.3 Review

9.4 Possible Extensions

Chapter 10: Project 7: Tilting Ball Maze

10.1 Overview

This project is a placeholder for Project 7 (Tilting Ball Maze). Students will use an MPU-6050 accelerometer/gyro and an OLED screen to draw a maze with a ball sprite that moves as the board is tilted.

10.2 Directions

10.2.1 Creating the circuit

10.2.2 Programming the microcontroller

Click on the file named "project_7_tilting_ball_maze.py" and run it using the IDE. Refer back to Chapter 3 for IDE instructions.

10.3 Review

10.4 Possible Extensions

Appendix A: Python Primer

If you're new to Python, this section will give you a few things you should know in order to better understand the projects in this guide. This is by no means a complete or comprehensive look at the Python language. For that, we recommend looking at the official Python site and reading through the tutorial there.

> Note: for the projects being used here, we are using an implementation of Python known as MicroPython. This version is meant to run on microcontrollers with limited resources. It also has built into it libraries for dealing with hardware devices that are not part of the standard CPython distribution. Therefore, not all Python examples you find online will run on your microcontroller and not all projects for a microcontroller can be run on your computer. But a lot of the code can be shared so the lessons you learn here can apply to other Python projects.

Here is a sample of a small Python script. We will disect and explain what each section does below:

Listing A.1: An example Python script

```
1  def show(message, repeat=1):
2      """This function prints the given message to the
3      console as smany times as specified in the
4      srepeat parameter.
5      """
6  
7      for iteration in range(0, repeat):
8          print(iteration, message)
9  
10 name = input("What is your name: ")
11 show(name)
12 show(name, repeat=3)
```

On line 1, we are defining a function named show. This function accepts two parameters, message and repeat. The message parameter is required and the repeat parameter is optional with a default value of 1.

Lines 2 through 5 comprise the docstring for the function. This information is meant for programmers to read and explains what the function does. It does not affect how the function works.

Line 7 starts a loop. The loop will repeat the statements in the loop body until a condition is met. In this case, it will loop until it has performed the operation for each repeat.

Line 8 is the body of the loop. This statement will print the message that the user passed in to the console along with the iteration number of the loop.

Line 10 prompts the user for their name and saves the result in a variable called name.

Line 11 calls our show function which will print the user's name once (the default).

Line 12 calls our show function again, this time saying that we want to repeat the loop of printing the name twice.

Running the program, we will see output like this:

```
$ python program.py
What is your name: Emily
0 Emily
0 Emily
1 Emily
2 Emily
$
```

Another feature that you'll see used often in Python are classes. Classes are a convienent way to model something in your program that holds state and implements functionality. For example, let's say that we are writing a game about racing go- karts. We need to allow each player to have their own kart and keep track of how fast it is going, which way they are turning, and allow the kart to speed up and slow down. Here is a small class that will help us do that:

Listing A.2: An example of a Python class

```
class Kart:
    MAXIMUM_SPEED = 100
```

```
3
4       def __init__(self):
5           """The kart starts motionless at the beginning"""
6           self._speed = 0
7           self._direction = 0
8           self._acceleration = 0
9
10      def brake(self):
11          """This is called when the user presses the brake button"""
12          self._acceleration = -5
13
14      def accelerate(self):
15          """This is called when the user presses the accelerator button"""
16          self._acceleration = 5
17
18      def steer(self, direction):
19          """This is called when the user presses left or right"""
20          self._direction = direction
21
22      def update(self, ticks):
23          """Update will be called by our game engine and will be
24          provided the number of ticks since it was last called.
25          """
26
27          self._speed += self._acceleration * ticks
28
29          # limit our speed so that we don't go faster than our
30          # kart is allowed to, or slower than 0
31          if self._speed > Kart.MAXIMUM_SPEED:
32              self._speed = Kart.MAXIMUM_SPEED
33          if self._speed < 0:
34              self._speed = 0
```

Looking at this class, there are 5 methods. The first one (on line 4) is a special method that is called by the Python interpreter whenever a new Kart is created. It will initialize some variables for this particular Kart object.

You may have noticed that the first method takes a parameter called "self". This is the first parameter of all methods in a class in Python. It is automatically passed by the interpreter and is a reference to the current object. It lets us access the variables that belong to the class, like those we defined in the __init__ method.

Speaking of the variables in the __init__ method. Notice how we named them all with an underscore? This is a convention in Python that says they are private to our class and that code written outside of the class shouldn't access them directly. That means that our class should provide ways to modify or read these variables via other methods.

The second method starts on line 10. This is called when the player presses the brake button on their controller and will set our Kart's acceleration to a negative value so that we start to slow down. It modifies the private _acceleration member of the class.

The third starts on line 14. It is the opposite of braking and will start speeding our Kart up when the user presses the accelerator. It aslo modifies the private _acceleration member of the class.

The fourth method, line 18, is again something to deal with user input. This time we can see that it takes a second parameter, direction. If the user presses left on their controller, then we can expect left to be passed here. The same for right. We will modify the private _direction member here.

Finally, we have a fifth method starting on line 22. This method is called by our game engine and uses the class members to determine what happens to the Kart throughout the game. That is, it is asking the Kart to update itself at a certain moment in time (usually once per frame) so that next time it draws it to the screen, it will be in the updated location.

Notice in the last method, we are accessing not only our own variables, _speed, and _acceleration, but we are also reading a class variable, Kart.MAXIMUM_SPEED. Unlike our member variables, a class variable is the same for all instances of a class. It is useful here to keep the game fair so that all Karts have the same limitation on their speed.